In [1]:
```
import pandas as pd
heart  = pd.read_csv('1645792390_cep1_dataset (1) - heart.csv')
heart
```

Out[1]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	63	1	3	145	233	1	0	150	0	2.3	0	0	1	1
1	37	1	2	130	250	0	1	187	0	3.5	0	0	2	1
2	41	0	1	130	204	0	0	172	0	1.4	2	0	2	1
3	56	1	1	120	236	0	1	178	0	0.8	2	0	2	1
4	57	0	0	120	354	0	1	163	1	0.6	2	0	2	1
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
298	57	0	0	140	241	0	1	123	1	0.2	1	0	3	0
299	45	1	3	110	264	0	1	132	0	1.2	1	0	3	0
300	68	1	0	144	193	1	1	141	0	3.4	1	2	3	0
301	57	1	0	130	131	0	1	115	1	1.2	1	1	3	0
302	57	0	1	130	236	0	0	174	0	0.0	1	1	2	0

303 rows × 14 columns

1a. Structure of the data

In [2]:
```
heart.shape
```

Out[2]:
```
(303, 14)
```

1b. Missing values in the data

In [4]:
```
heart.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 303 entries, 0 to 302
Data columns (total 14 columns):
 #   Column    Non-Null Count  Dtype  
---  ------    --------------  -----  
 0   age       303 non-null    int64  
 1   sex       303 non-null    int64  
 2   cp        303 non-null    int64  
 3   trestbps  303 non-null    int64  
 4   chol      303 non-null    int64  
 5   fbs       303 non-null    int64  
 6   restecg   303 non-null    int64  
 7   thalach   303 non-null    int64  
 8   exang     303 non-null    int64  
 9   oldpeak   303 non-null    float64
 10  slope     303 non-null    int64  
 11  ca        303 non-null    int64  
 12  thal      303 non-null    int64  
 13  target    303 non-null    int64  
dtypes: float64(1), int64(13)
memory usage: 33.3 KB
```

In [5]:
```
heart.isna().sum()
```

Out[5]:
```
age         0
sex         0
cp          0
trestbps    0
chol        0
fbs         0
restecg     0
thalach     0
exang       0
oldpeak     0
slope       0
ca          0
thal        0
target      0
dtype: int64
```

In [6]:
```
heart.duplicated()
```

Out[6]:
```
0      False
1      False
2      False
3      False
4      False
       ...
298    False
299    False
300    False
301    False
302    False
Length: 303, dtype: bool
```

2a. Statistical summary of the dataset

In [7]:
```
heart[['age', 'trestbps', 'chol', 'thalach', 'oldpeak']].describe()
```

Out[7]:

	age	trestbps	chol	thalach	oldpeak
count	303.000000	303.000000	303.000000	303.000000	303.000000
mean	54.366337	131.623762	246.264026	149.646865	1.039604
std	9.082101	17.538143	51.830751	22.905161	1.161075
min	29.000000	94.000000	126.000000	71.000000	0.000000
25%	47.500000	120.000000	211.000000	133.500000	0.000000
50%	55.000000	130.000000	240.000000	153.000000	0.800000
75%	61.000000	140.000000	274.500000	166.000000	1.600000
max	77.000000	200.000000	564.000000	202.000000	6.200000

2b. Plotting the categorical variables of the dataset

In [8]:
```
df = heart.iloc[:, [1,2,5,6,8,10,11,12,13]]
df
```

Out[8]:

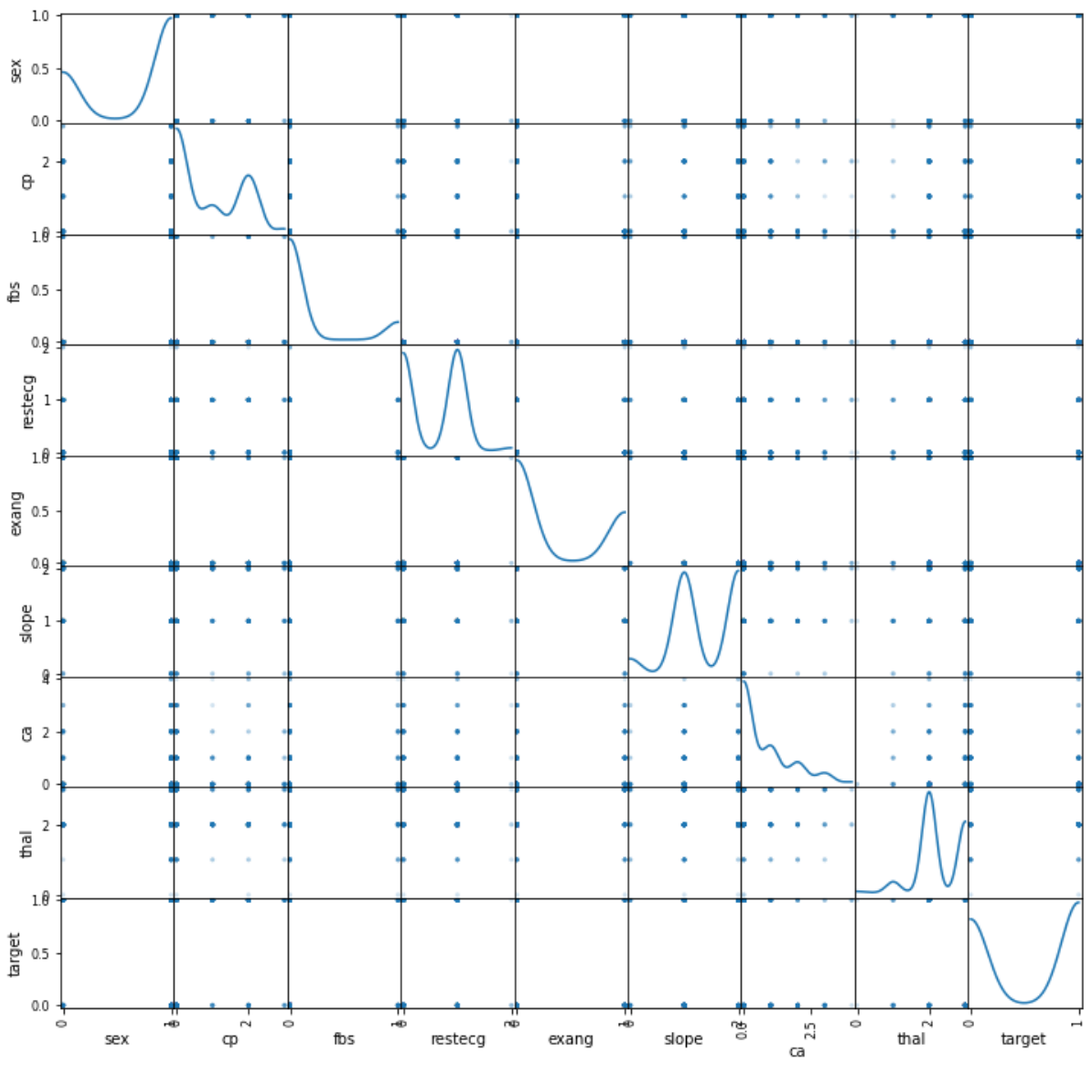
	sex	cp	fbs	restecg	exang	slope	ca	thal	target
0	1	3	1	0	0	0	0	1	1
1	1	2	0	1	0	0	0	2	1
2	0	1	0	0	0	2	0	2	1
3	1	1	0	1	0	2	0	2	1
4	0	0	0	1	1	2	0	2	1
...	...	...	...	...	...	...	...	...	...
298	0	0	0	1	1	1	0	3	0
299	1	3	0	1	0	1	0	3	0
300	1	0	1	1	0	1	2	3	0
301	1	0	0	1	1	1	1	3	0
302	0	1	0	0	0	1	1	2	0

303 rows × 9 columns

In [9]:
```
from pandas.plotting import scatter_matrix
scatter_matrix(df, alpha = 0.2, figsize = (12,12), diagonal = 'kde')
```

Out[9]:
```
array([[<matplotlib.axes._subplots.AxesSubplot object at 0x00000230819AF190>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x00000230820B0550>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x00000230820DC9A0>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x000002308210AE20>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x00000230821452B0>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x000002308216E640>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x000002308216E730>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x0000023082198BE0>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x00000230821FD430
>],
       [<matplotlib.axes._subplots.AxesSubplot object at 0x000002308222C880>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x0000023082258D00>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x0000023082291190>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x00000230822BC5E0>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x00000230822E7A30>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x0000023082313E80>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x000002308234A340>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x00000230823787C0>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x00000230823A2C10
>],
       [<matplotlib.axes._subplots.AxesSubplot object at 0x00000230823D1100>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x00000230824094F0>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x0000023082435940>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x0000023082462DF0>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x000002308249A280>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x00000230824C7730>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x00000230824F2B80>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x000002308251EFD0>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x0000023082557460
>],
       [<matplotlib.axes._subplots.AxesSubplot object at 0x00000230825828B0>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x00000230825B1D00>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x00000230825E91F0>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x0000023082610D90>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x0000023082117C70>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x0000023082241D00>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x0000023082383DC0>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x00000230824C7F10>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x0000023082622D60
>],
       [<matplotlib.axes._subplots.AxesSubplot object at 0x000002308271A250>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x00000230827466A0>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x0000023082772AF0>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x00000230827A0F40>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x00000230827D83D0>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x0000023082805820>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x000002308282EC70>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x000002308285E160>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x0000023082895550
>],
       [<matplotlib.axes._subplots.AxesSubplot object at 0x00000230828C09A0>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x00000230828EDDF0>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x0000023082925280>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x00000230829526D0>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x000002308297EB20>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x00000230829ABF70>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x00000230829E2400>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x0000023082A0E850>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x0000023082A3DCA0
>],
       [<matplotlib.axes._subplots.AxesSubplot object at 0x0000023082A76130>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x0000023082AA3580>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x0000023082ACE9D0>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x0000023082AFBE80>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x0000023082B32310>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x0000023082B5E760>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x0000023082B8CBB0>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x0000023082BBBAC0>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x0000023082BF2280
>],
       [<matplotlib.axes._subplots.AxesSubplot object at 0x0000023082C19A00>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x0000023082C4E2B0>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x0000023082C76A30>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x0000023082CAB1F0>,
```

```
        <matplotlib.axes._subplots.AxesSubplot object at 0x0000023082CD4970>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x0000023082D0B130>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x0000023082D348E0>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x0000023082D5E100>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x0000023082D92820
>],
       [<matplotlib.axes._subplots.AxesSubplot object at 0x0000023082DBBFA0>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x0000023082DF1760>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x0000023082E18F70>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x0000023082E4D730>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x0000023083E48EB0>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x0000023083E7E670>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x0000023083EA4DF0>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x0000023083EDA5B0>,
        <matplotlib.axes._subplots.AxesSubplot object at 0x0000023083F03D30
>]],
      dtype=object)
```



In [10]:
```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler(copy = True, with_mean = True, with_std = True)
samples_scaled = scaler.fit_transform(heart)
print(samples_scaled)
```

```
[[ 0.9521966   0.68100522  1.97312292 ... -0.71442887 -2.14887271
   0.91452919]
 [-1.91531289  0.68100522  1.00257707 ... -0.71442887 -0.51292188
   0.91452919]
 [-1.47415758 -1.46841752  0.03203122 ... -0.71442887 -0.51292188
   0.91452919]
 ...
 [ 1.50364073  0.68100522 -0.93851463 ...  1.24459328  1.12302895
  -1.09345881]
 [ 0.29046364  0.68100522 -0.93851463 ...  0.26508221  1.12302895
  -1.09345881]
 [ 0.29046364 -1.46841752  0.03203122 ...  0.26508221 -0.51292188
  -1.09345881]]
```

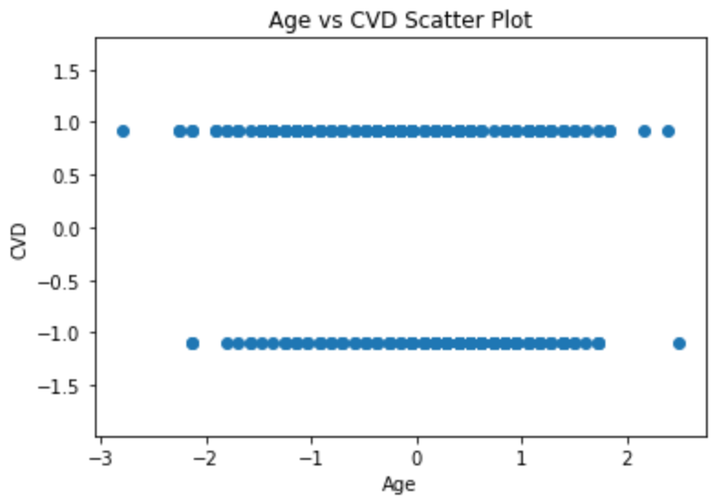
2c. Occurence of CVD across the Age category

In [13]:
```
age = samples_scaled[:,0]
target = samples_scaled[:,13]

import matplotlib.pyplot as plt
plt.scatter(age, target)
plt.axis('equal')
plt.xlabel('Age')
plt.ylabel('CVD')
plt.title('Age vs CVD Scatter Plot')
plt.show

from scipy.stats import pearsonr
correlation, pvalue = pearsonr(age, target)
print(correlation)
```

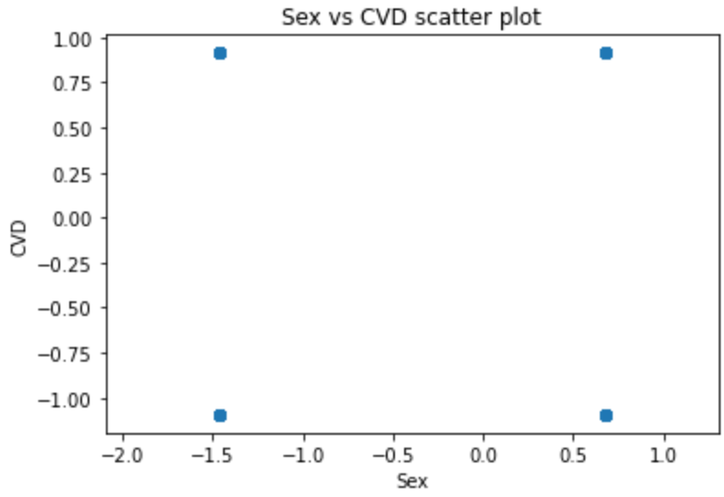
```
-0.22543871587483733
```



2d. Occurence of CVD across Sex category

In [19]:
```
sex = samples_scaled[:,1]
target = samples_scaled[:,13]
plt.scatter(sex, target)
plt.xlabel('Sex')
plt.ylabel('CVD')
plt.title('Sex vs CVD scatter plot')
plt.axis('equal')
plt.show()

correlation, pvalue = pearsonr(sex,target)
print(correlation)
```

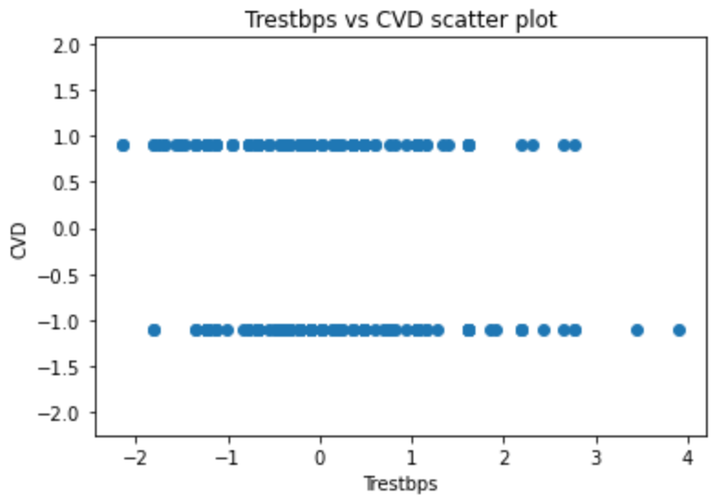



```
-0.280936575501767
```

2e. trestbps vs CVD

In [20]:
```
trestbps = samples_scaled[:,3]
target = samples_scaled[:,13]
plt.scatter(trestbps, target)
plt.xlabel('Trestbps')
plt.ylabel('CVD')
plt.title('Trestbps vs CVD scatter plot')
plt.axis('equal')
plt.show()

correlation, pvalue = pearsonr(trestbps,target)
print(correlation)
```

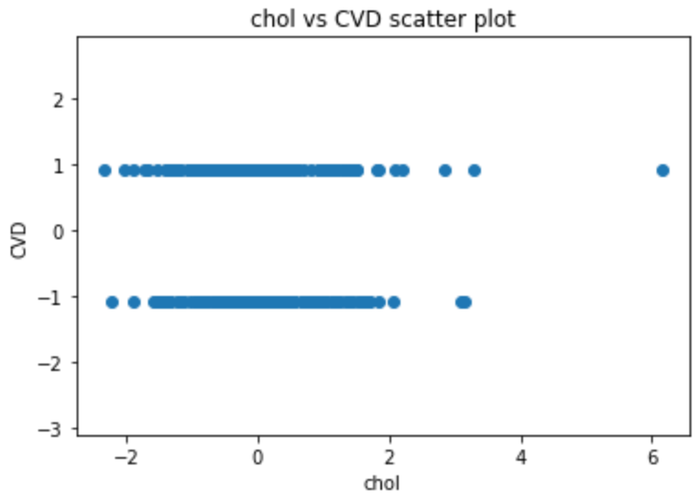



```
-0.1449311284977516
```

2f. Cholesterol vs Target

In [21]:
```
chol = samples_scaled[:,4]
target = samples_scaled[:,13]
plt.scatter(chol, target)
plt.xlabel('chol')
plt.ylabel('CVD')
plt.title('chol vs CVD scatter plot')
plt.axis('equal')
plt.show()

correlation, pvalue = pearsonr(chol,target)
print(correlation)
```

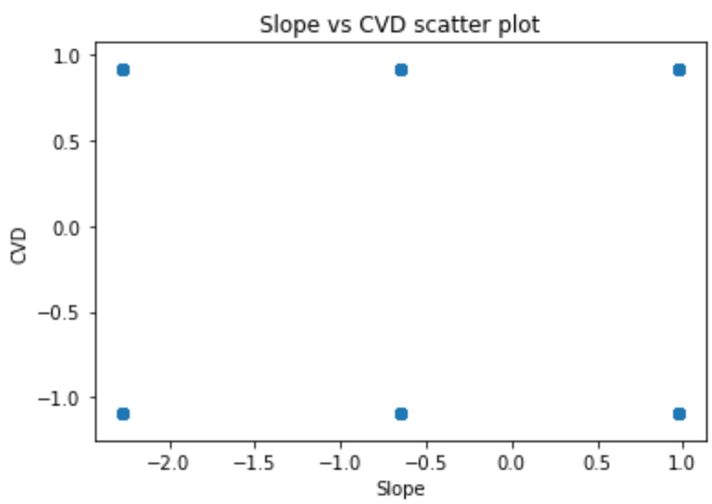



```
-0.08523910513756906
```

2g. Peak exercise(slope) vs CVD

In [22]:
```
slope = samples_scaled[:,10]
target = samples_scaled[:,13]
plt.scatter(slope, target)
plt.xlabel('Slope')
plt.ylabel('CVD')
plt.title('Slope vs CVD scatter plot')
plt.axis('equal')
plt.show()

correlation, pvalue = pearsonr(slope,target)
print(correlation)
```

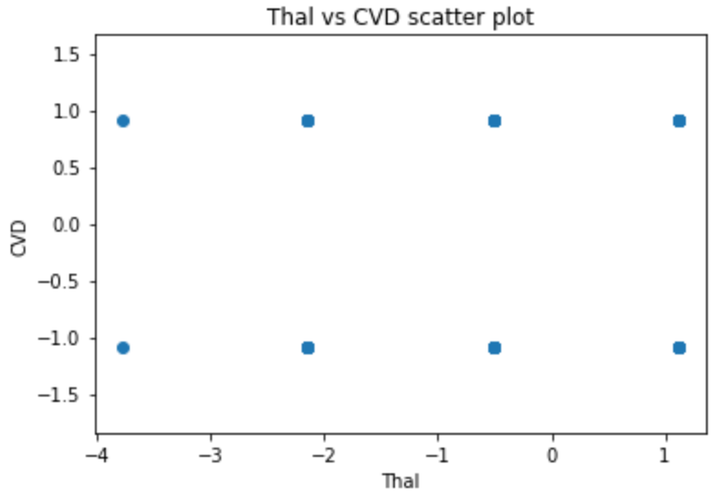



```
0.34587707824172487
```

2h. Thallassemia vs CVD

In [23]:
```
thal = samples_scaled[:,12]
target = samples_scaled[:,13]
plt.scatter(thal, target)
plt.xlabel('Thal')
plt.ylabel('CVD')
plt.title('Thal vs CVD scatter plot')
plt.axis('equal')
plt.show()

correlation, pvalue = pearsonr(thal,target)
print(correlation)
```




```
-0.34402926803831013
```

2i. All factors heatmap

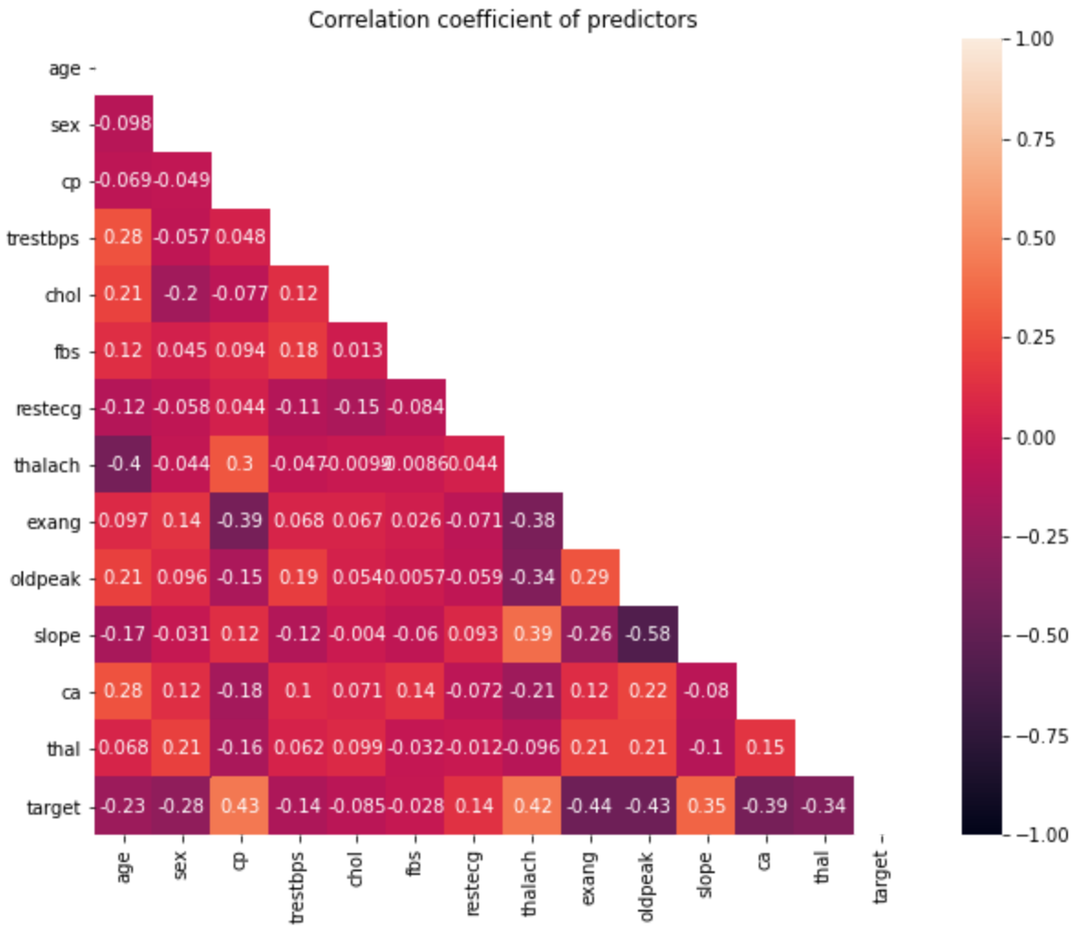
In [24]:
```
heart.corr()
```

Out[24]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	
age	1.000000	-0.098447	-0.068653	0.279351	0.213678	0.121308	-0.116211	-0.398522	0
sex	-0.098447	1.000000	-0.049353	-0.056769	-0.197912	0.045032	-0.058196	-0.044020	0
cp	-0.068653	-0.049353	1.000000	0.047608	-0.076904	0.094444	0.044421	0.295762	-0
trestbps	0.279351	-0.056769	0.047608	1.000000	0.123174	0.177531	-0.114103	-0.046698	0
chol	0.213678	-0.197912	-0.076904	0.123174	1.000000	0.013294	-0.151040	-0.009940	0
fbs	0.121308	0.045032	0.094444	0.177531	0.013294	1.000000	-0.084189	-0.008567	0
restecg	-0.116211	-0.058196	0.044421	-0.114103	-0.151040	-0.084189	1.000000	0.044123	-0
thalach	-0.398522	-0.044020	0.295762	-0.046698	-0.009940	-0.008567	0.044123	1.000000	-0
exang	0.096801	0.141664	-0.394280	0.067616	0.067023	0.025665	-0.070733	-0.378812	1
oldpeak	0.210013	0.096093	-0.149230	0.193216	0.053952	0.005747	-0.058770	-0.344187	0
slope	-0.168814	-0.030711	0.119717	-0.121475	-0.004038	-0.059894	0.093045	0.386784	-0
ca	0.276326	0.118261	-0.181053	0.101389	0.070511	0.137979	-0.072042	-0.213177	0
thal	0.068001	0.210041	-0.161736	0.062210	0.098803	-0.032019	-0.011981	-0.096439	0
target	-0.225439	-0.280937	0.433798	-0.144931	-0.085239	-0.028046	0.137230	0.421741	-0

In [25]:
```
import numpy as np
import seaborn as sns

plt.figure(figsize = (10,8))
mask = np.triu(np.ones_like(heart.corr(), dtype = bool))
sns.heatmap(heart.corr(), annot = True, mask = mask, vmin = -1, vmax = 1)
plt.title('Correlation coefficient of predictors')
plt.show()
```



3a. Logistic Regression

In [26]:
```
heart
```

Out[26]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	63	1	3	145	233	1	0	150	0	2.3	0	0	1	1
1	37	1	2	130	250	0	1	187	0	3.5	0	0	2	1
2	41	0	1	130	204	0	0	172	0	1.4	2	0	2	1
3	56	1	1	120	236	0	1	178	0	0.8	2	0	2	1
4	57	0	0	120	354	0	1	163	1	0.6	2	0	2	1
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
298	57	0	0	140	241	0	1	123	1	0.2	1	0	3	0
299	45	1	3	110	264	0	1	132	0	1.2	1	0	3	0
300	68	1	0	144	193	1	1	141	0	3.4	1	2	3	0
301	57	1	0	130	131	0	1	115	1	1.2	1	1	3	0
302	57	0	1	130	236	0	0	174	0	0.0	1	1	2	0

303 rows × 14 columns

In [27]:
```
x = heart.drop('target', axis = 1)
y = heart['target']
```

In [28]:
```
x.shape
```

Out[28]:
```
(303, 13)
```

In [29]:
```
y.shape
```

Out[29]:
```
(303,)
```

In [30]:
```
x['cp'].unique()
```

Out[30]:
```
array([3, 2, 1, 0], dtype=int64)
```

In [31]:
```
x['restecg'].unique()
```

Out[31]:
```
array([0, 1, 2], dtype=int64)
```

In [32]:
```
x['slope'].unique()
```

Out[32]:
```
array([0, 2, 1], dtype=int64)
```

In [33]:
```
x['ca'].unique()
```

Out[33]:
```
array([0, 2, 1, 3, 4], dtype=int64)
```

In [34]:
```
x['thal'].unique()
```

Out[34]:
```
array([1, 2, 3, 0], dtype=int64)
```

In [35]:
```
# One- Hot Encoding
x_encoded = pd.get_dummies(x, columns = ['cp', 'restecg', 'slope', 'ca', 'tha
l'])
x_encoded
```

Out[35]:

	age	sex	trestbps	chol	fbs	thalach	exang	oldpeak	cp_0	cp_1	...	slope_2	ca_0	ca_
0	63	1	145	233	1	150	0	2.3	0	0	...	0	1	(
1	37	1	130	250	0	187	0	3.5	0	0	...	0	1	(
2	41	0	130	204	0	172	0	1.4	0	1	...	1	1	(
3	56	1	120	236	0	178	0	0.8	0	1	...	1	1	(
4	57	0	120	354	0	163	1	0.6	1	0	...	1	1	(
...	...	...	...	...	...	...	...	...	...	...	...	...	...	..
298	57	0	140	241	0	123	1	0.2	1	0	...	0	1	(
299	45	1	110	264	0	132	0	1.2	0	0	...	0	1	(
300	68	1	144	193	1	141	0	3.4	1	0	...	0	0	(
301	57	1	130	131	0	115	1	1.2	1	0	...	0	0	'
302	57	0	130	236	0	174	0	0.0	0	1	...	0	0	'

303 rows × 27 columns

In [36]:
```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
```

In [37]:
```
x_train, x_test, y_train, y_test = train_test_split(x_encoded, y, stratify =
y, test_size = 0.25, random_state = 123)
```

In [38]:
```
logreg = LogisticRegression(max_iter = 200)
logreg
```

Out[38]:
```
LogisticRegression(max_iter=200)
```

In [39]:
```
logreg.fit(x_train, y_train)
```

```
C:\ProgramData\Anaconda3\lib\site-packages\sklearn\linear_model\_logistic.py:
762: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regres
sion
  n_iter_i = _check_optimize_result(
```

Out[39]:
```
LogisticRegression(max_iter=200)
```

In [40]:
```
y_pred = logreg.predict(x_test)
```

In [41]:
```
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
[[27  8]
 [ 5 36]]
              precision    recall  f1-score   support

           0       0.84      0.77      0.81        35
           1       0.82      0.88      0.85        41

    accuracy                           0.83        76
   macro avg       0.83      0.82      0.83        76
weighted avg       0.83      0.83      0.83        76
```

In [42]:
```
# Checking the overfitting
y_pred_train = logreg.predict(x_train)
print(confusion_matrix(y_train, y_pred_train))
print(classification_report(y_train, y_pred_train))
```

```
[[ 86  17]
 [  9 115]]
              precision    recall  f1-score   support

           0       0.91      0.83      0.87       103
           1       0.87      0.93      0.90       124

    accuracy                           0.89       227
   macro avg       0.89      0.88      0.88       227
weighted avg       0.89      0.89      0.88       227
```

Test accuracy = 83%. Train Accuracy = 89% Occurence of Overfitting

In [44]:
```
import statsmodels.formula.api as smf
```

In [45]:
```
log_reg = smf.logit('target ~ age + sex + cp + trestbps + chol + fbs + restecg
+ thalach + exang + oldpeak + slope + ca + thal', data = heart).fit()
```

```
Optimization terminated successfully.
         Current function value: 0.348904
         Iterations 7
```

Results

In [46]:
```
print(log_reg.summary())
```

```
                           Logit Regression Results
==============================================================================
Dep. Variable:                 target   No. Observations:                  303
Model:                          Logit   Df Residuals:                      289
Method:                           MLE   Df Model:                           13
Date:                Wed, 29 Nov 2023   Pseudo R-squ.:                  0.4937
Time:                        01:28:47   Log-Likelihood:                -105.72
converged:                       True   LL-Null:                       -208.82
Covariance Type:            nonrobust   LLR p-value:                 7.262e-37
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept      3.4505      2.571      1.342      0.180      -1.590       8.490
age           -0.0049      0.023     -0.212      0.832      -0.050       0.041
sex           -1.7582      0.469     -3.751      0.000      -2.677      -0.839
cp             0.8599      0.185      4.638      0.000       0.496       1.223
trestbps      -0.0195      0.010     -1.884      0.060      -0.040       0.001
chol          -0.0046      0.004     -1.224      0.221      -0.012       0.003
fbs            0.0349      0.529      0.066      0.947      -1.003       1.073
restecg        0.4663      0.348      1.339      0.181      -0.216       1.149
thalach        0.0232      0.010      2.219      0.026       0.003       0.044
exang         -0.9800      0.410     -2.391      0.017      -1.783      -0.177
oldpeak       -0.5403      0.214     -2.526      0.012      -0.959      -0.121
slope          0.5793      0.350      1.656      0.098      -0.106       1.265
ca            -0.7733      0.191     -4.051      0.000      -1.147      -0.399
thal          -0.9004      0.290     -3.104      0.002      -1.469      -0.332
==============================================================================
```

In [47]:
```
log_reg_new = smf.logit('target ~ sex + cp + trestbps + thalach + exang + oldp
eak + slope + ca + thal', data = heart).fit()
print(log_reg_new.summary())
```

```
Optimization terminated successfully.
         Current function value: 0.356051
         Iterations 7
                           Logit Regression Results
==============================================================================
=
Dep. Variable:                 target   No. Observations:                  30
3
Model:                          Logit   Df Residuals:                      29
3
Method:                           MLE   Df Model:
9
Date:                Wed, 29 Nov 2023   Pseudo R-squ.:                   0.483
4
Time:                        01:29:21   Log-Likelihood:                -107.8
8
converged:                       True   LL-Null:                       -208.8
2
Covariance Type:            nonrobust   LLR p-value:                 1.343e-3
8
==============================================================================
=
                 coef    std err          z      P>|z|      [0.025      0.97
5]
------------------------------------------------------------------------------
-
Intercept      2.4234      1.916      1.265      0.206      -1.332       6.17
9
sex           -1.5888      0.433     -3.667      0.000      -2.438      -0.74
0
cp             0.8541      0.180      4.739      0.000       0.501       1.20
7
trestbps      -0.0210      0.010     -2.131      0.033      -0.040      -0.00
2
thalach        0.0228      0.009      2.451      0.014       0.005       0.04
1
exang         -0.9472      0.401     -2.364      0.018      -1.732      -0.16
2
oldpeak       -0.5313      0.207     -2.562      0.010      -0.938      -0.12
5
slope          0.6045      0.341      1.770      0.077      -0.065       1.27
4
ca            -0.7553      0.184     -4.115      0.000      -1.115      -0.39
6
thal          -0.9160      0.279     -3.278      0.001      -1.464      -0.36
8
==============================================================================
=
```

In [48]:
```
print(log_reg_new.params)
```

```
Intercept    2.423391
sex         -1.588807
cp           0.854141
trestbps    -0.021043
thalach      0.022843
exang       -0.947169
oldpeak     -0.531327
slope        0.604485
ca          -0.755279
thal        -0.916021
dtype: float64
```

3b. Random Forest

In [49]:
```
x_train, x_test, y_train, y_test = train_test_split(x_encoded, y, random_state = 42)
```

In [50]:
```
from sklearn.ensemble import RandomForestClassifier
```

In [51]:
```
rf = RandomForestClassifier(random_state = 42)
params = {
    'max_depth' : [2,3],
    'min_samples_leaf' : range(5,8),
    'n_estimators' : range(25,30)
}
```

In [52]:
```
from sklearn.model_selection import GridSearchCV
grid_search = GridSearchCV(estimator = rf, param_grid = params, cv = 4, scoring = 'accuracy')
```

In [53]:
```
grid_search.fit(x_train, y_train)
```

Out[53]:
```
GridSearchCV(cv=4, estimator=RandomForestClassifier(random_state=42),
             param_grid={'max_depth': [2, 3], 'min_samples_leaf': range(5, 8),
                         'n_estimators': range(25, 30)},
             scoring='accuracy')
```

In [54]:
```
rf_best = grid_search.best_estimator_
rf_best
```

Out[54]:
```
RandomForestClassifier(max_depth=3, min_samples_leaf=6, n_estimators=25,
                       random_state=42)
```

In [55]:
```
y_pred = rf_best.predict(x_test)
```

In [56]:
```
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
[[29  6]
 [ 4 37]]
              precision    recall  f1-score   support

           0       0.88      0.83      0.85        35
           1       0.86      0.90      0.88        41

    accuracy                           0.87        76
   macro avg       0.87      0.87      0.87        76
weighted avg       0.87      0.87      0.87        76
```

In [57]:
```
# Checking the overfitting
y_pred_train = rf_best.predict(x_train)
print(confusion_matrix(y_train, y_pred_train))
print(classification_report(y_train, y_pred_train))
```

```
[[ 82  21]
 [ 10 114]]
              precision    recall  f1-score   support

           0       0.89      0.80      0.84       103
           1       0.84      0.92      0.88       124

    accuracy                           0.86       227
   macro avg       0.87      0.86      0.86       227
weighted avg       0.87      0.86      0.86       227
```

Test accuracy = 87% Train accuracy = 86% No overfitting in the model

In [58]:
```
rf_best.feature_importances_
```

Out[58]:
```
array([0.03945581, 0.01622971, 0.0253221 , 0.01567825, 0.        ,
       0.0732047 , 0.13065256, 0.08844702, 0.04767619, 0.0083345 ,
       0.03263172, 0.00098566, 0.00516403, 0.00474132, 0.        ,
       0.        , 0.0282683 , 0.01529171, 0.13734364, 0.02591035,
       0.01236833, 0.00884048, 0.        , 0.        , 0.        ,
       0.18421251, 0.0992411 ])
```

Feature significance

In [59]:
```
imp_df = pd.DataFrame({
    "Varname": x_train.columns,
    "Imp": rf_best.feature_importances_
})
imp_df
```

Out[59]:

	Varname	Imp
0	age	0.039456
1	sex	0.016230
2	trestbps	0.025322
3	chol	0.015678
4	fbs	0.000000
5	thalach	0.073205
6	exang	0.130653
7	oldpeak	0.088447
8	cp_0	0.047676
9	cp_1	0.008334
10	cp_2	0.032632
11	cp_3	0.000986
12	restecg_0	0.005164
13	restecg_1	0.004741
14	restecg_2	0.000000
15	slope_0	0.000000
16	slope_1	0.028268
17	slope_2	0.015292
18	ca_0	0.137344
19	ca_1	0.025910
20	ca_2	0.012368
21	ca_3	0.008840
22	ca_4	0.000000
23	thal_0	0.000000
24	thal_1	0.000000
25	thal_2	0.184213
26	thal_3	0.099241

In [60]:
```
imp_df.sort_values(by="Imp", ascending=False)
```

Out[60]:

	Varname	Imp
25	thal_2	0.184213
18	ca_0	0.137344
6	exang	0.130653
26	thal_3	0.099241
7	oldpeak	0.088447
5	thalach	0.073205
8	cp_0	0.047676
0	age	0.039456
10	cp_2	0.032632
16	slope_1	0.028268
19	ca_1	0.025910
2	trestbps	0.025322
1	sex	0.016230
3	chol	0.015678
17	slope_2	0.015292
20	ca_2	0.012368
21	ca_3	0.008840
9	cp_1	0.008334
12	restecg_0	0.005164
13	restecg_1	0.004741
11	cp_3	0.000986
15	slope_0	0.000000
14	restecg_2	0.000000
22	ca_4	0.000000
23	thal_0	0.000000
24	thal_1	0.000000
4	fbs	0.000000

In [ ]:
```

```